Hardware-Software Interface

Tutorial: Pointer Exercises

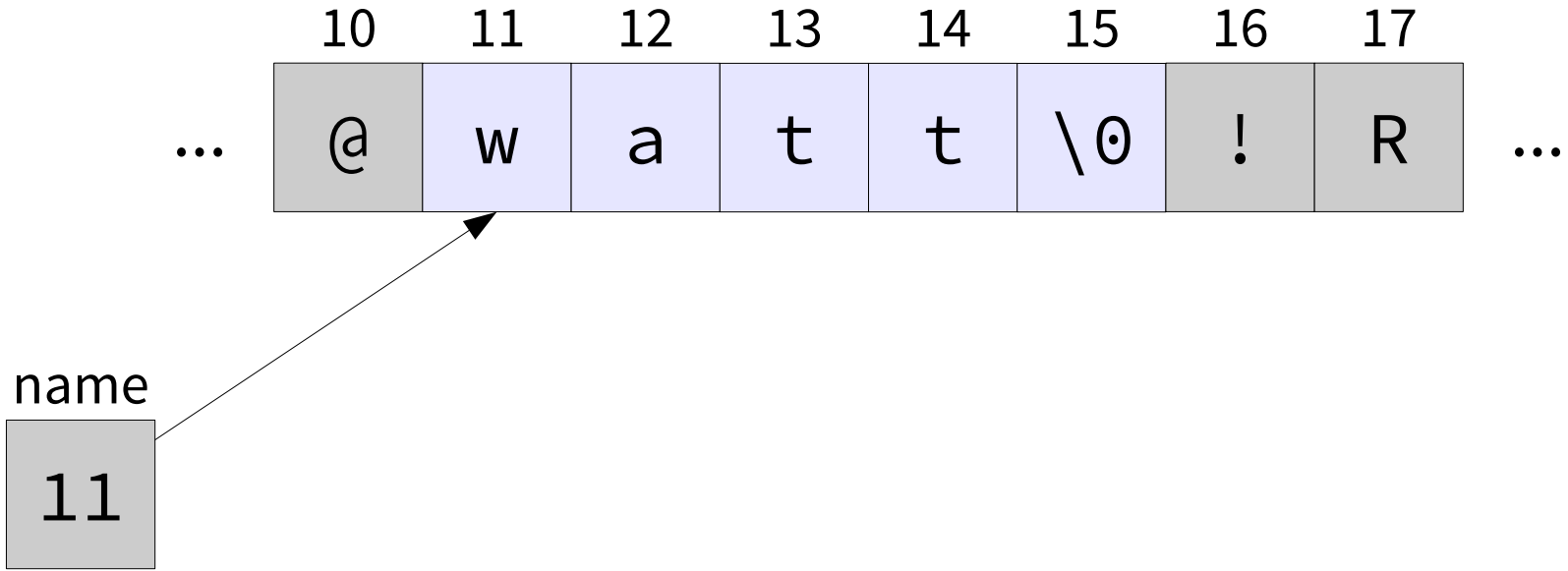
Adam Sampson
School of Mathematical and Computer Sciences
Heriot-Watt University

A C string in memory

	10	11	12	13	14	15	16	17	
…	@	w	a	t	t	\0	!	R	…

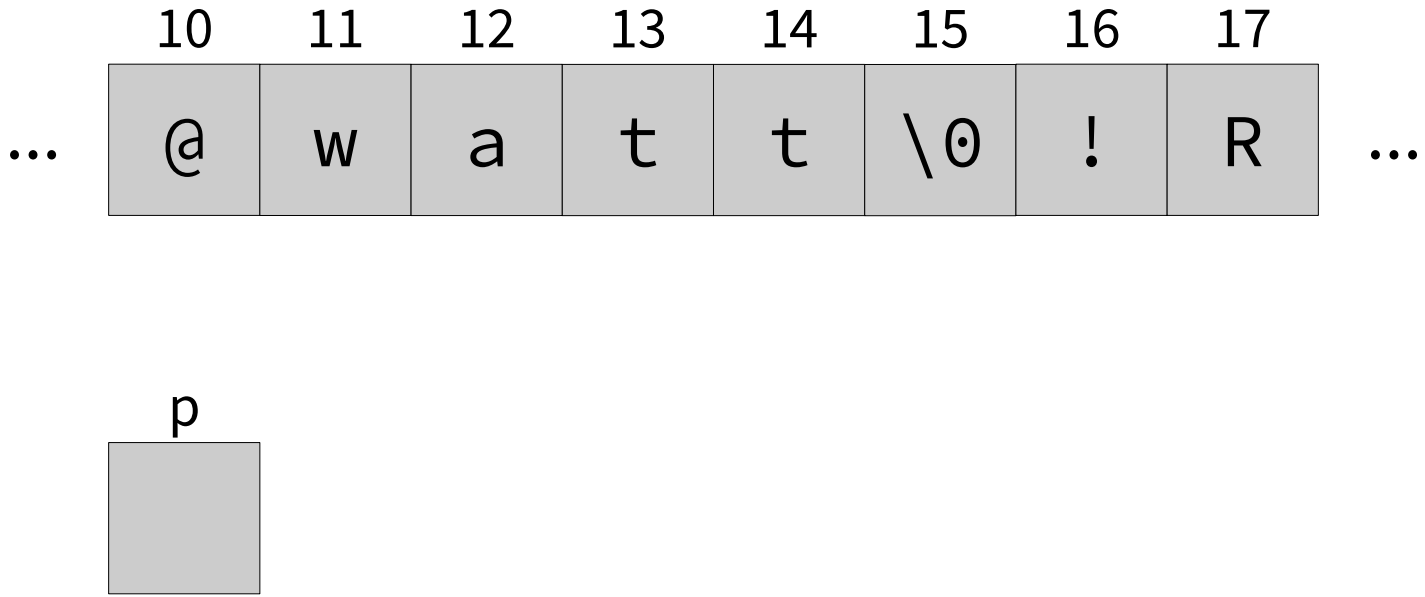
```
char name[] = “watt”;
```

Arrays are pointers



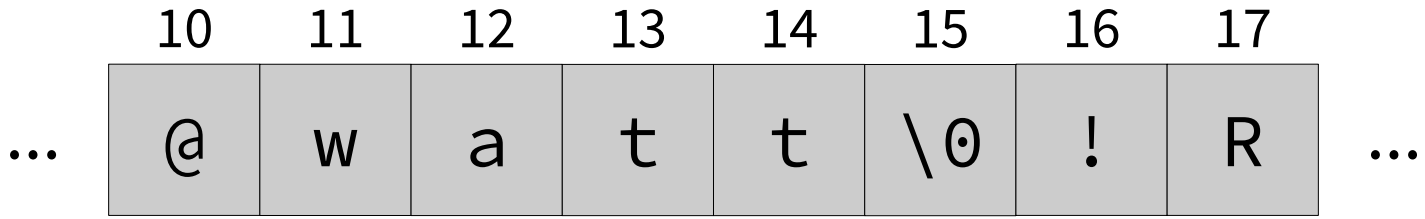

```
char name[] = “watt”;
```

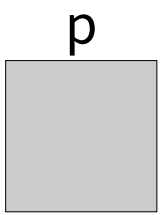
A pointer




```
char *p;
```

Q1



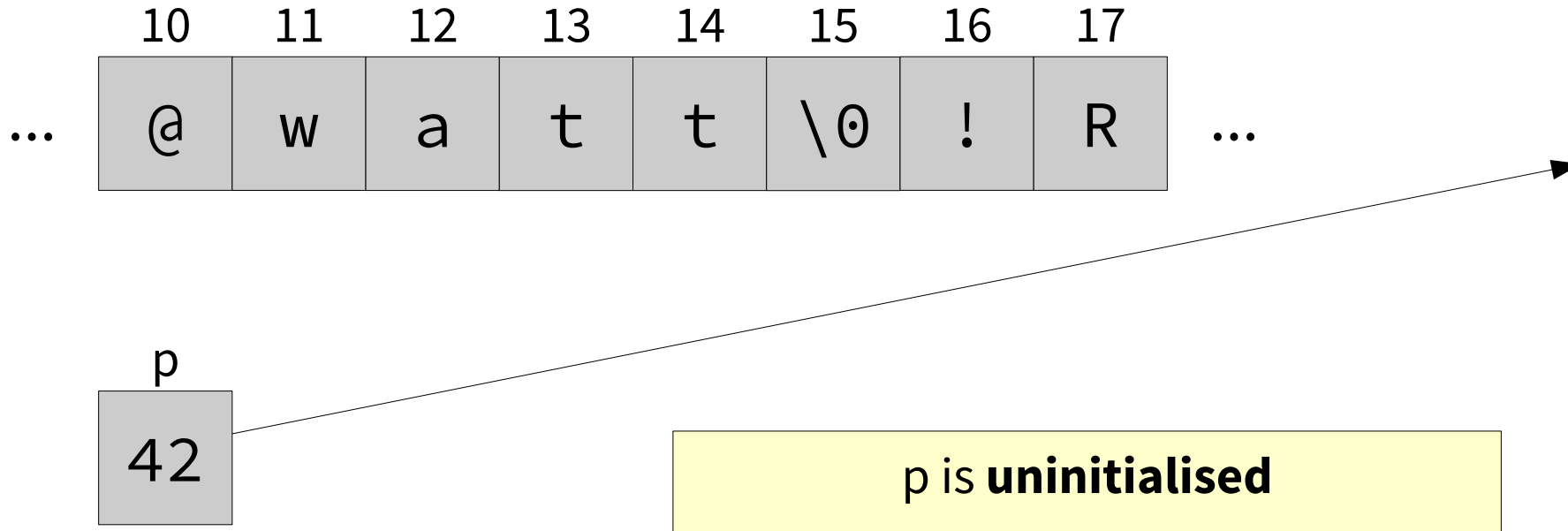



```
char *p;
```

What does p point to?

(or: what value does p have?)

A1

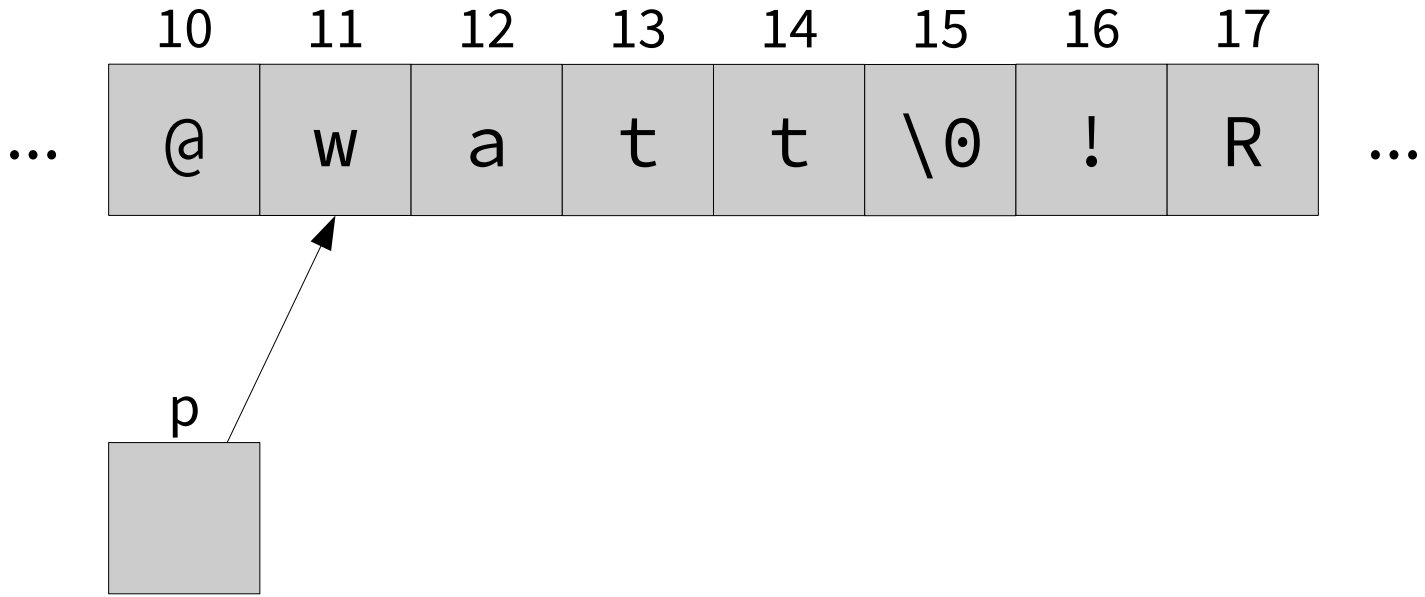



```
char *p;
```

p is **uninitialised**

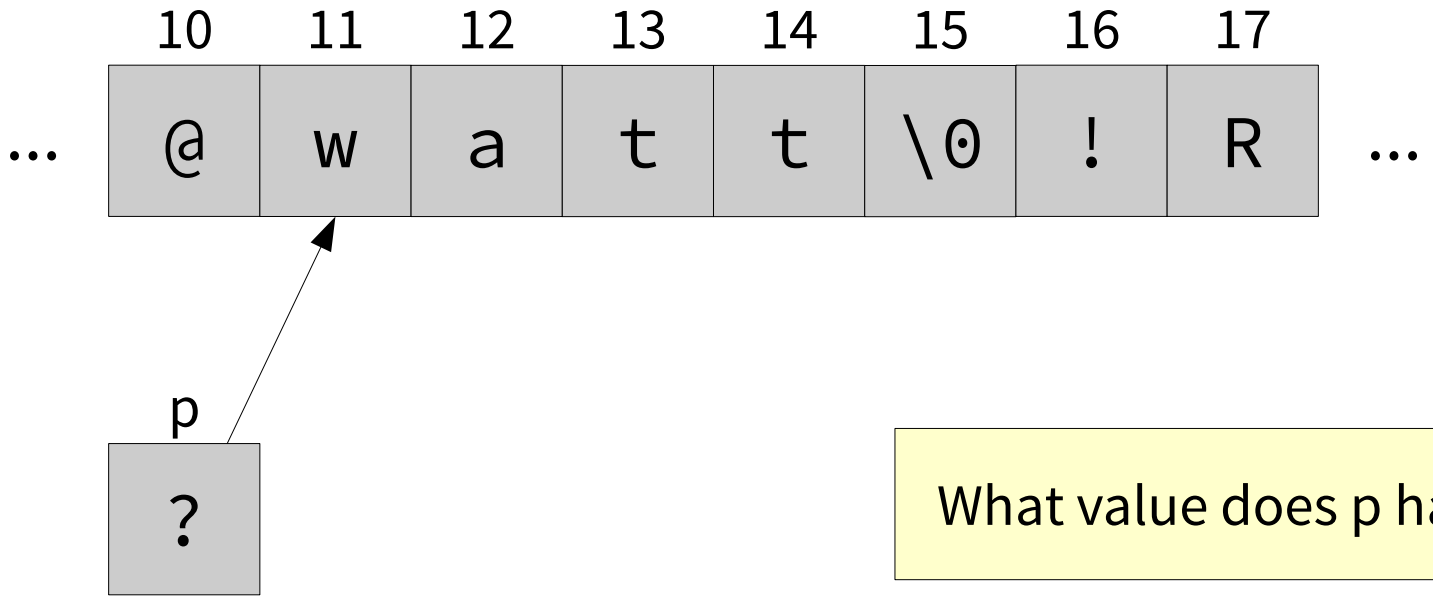
It will have some unknown value (e.g. whatever was in memory before), so we don't know.

Initialising the pointer




```
char name[] = “watt”;
char *p = name;
```

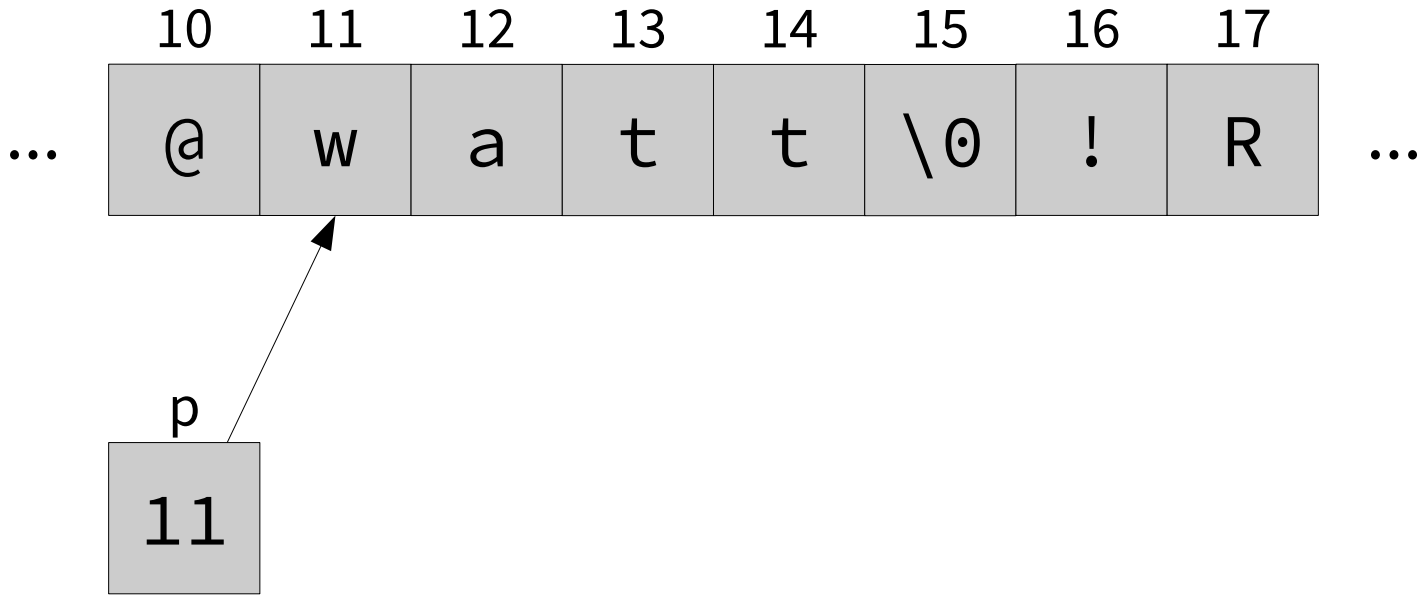
Q2



What value does p have now?

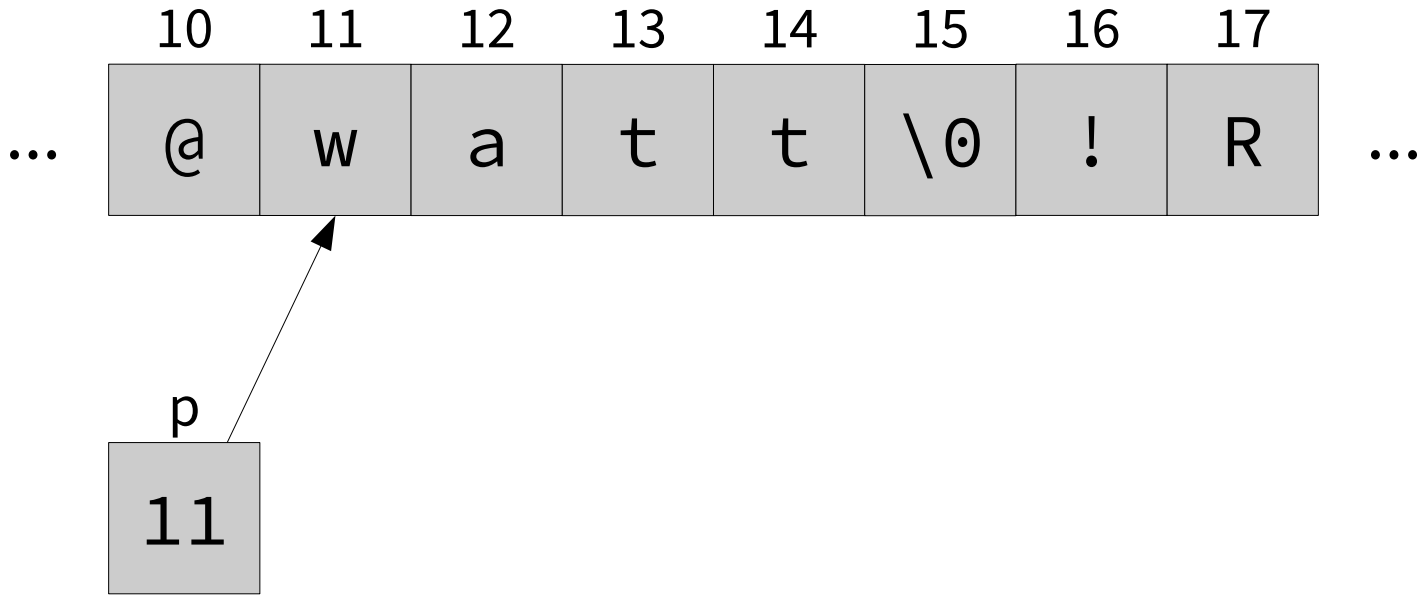
```
char name[] = “watt”;
char *p = name;
```

A2



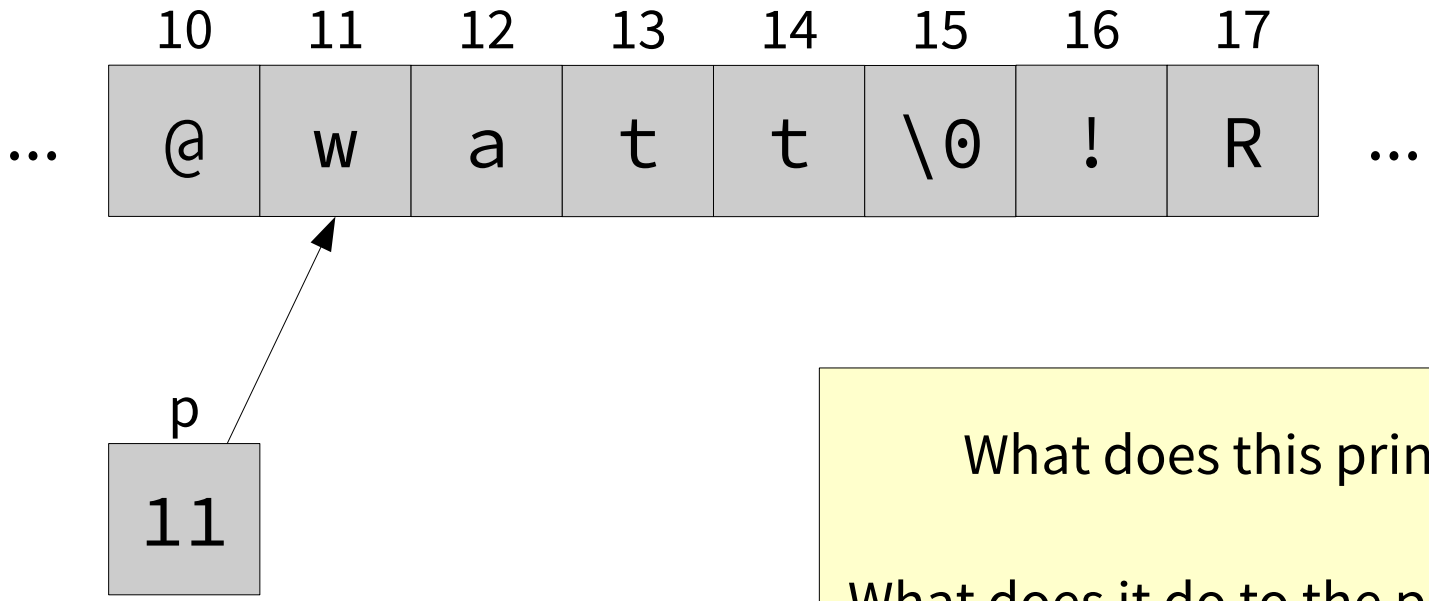

```
char name[] = “watt”;
char *p = name;
```

Printing chars




```
printf("%c", 'X');
   // prints X (%c = character)
```

Q3

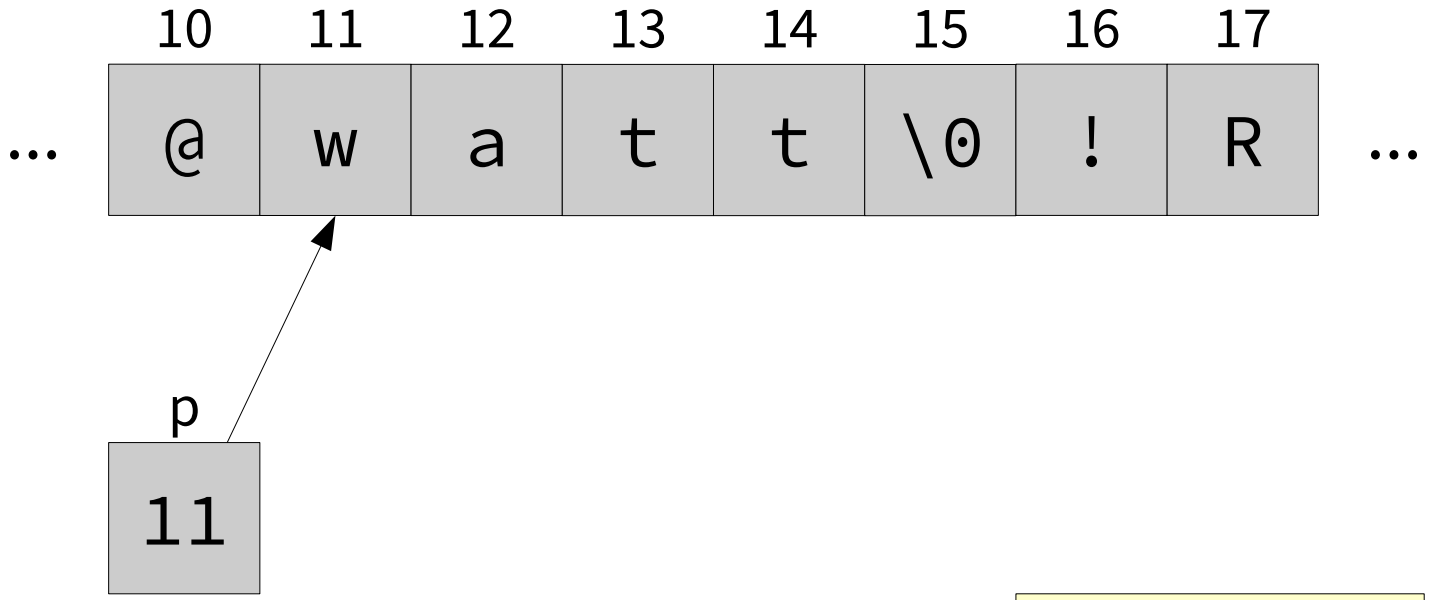


What does this print?

What does it do to the picture?

```
printf("%c", *p);
```

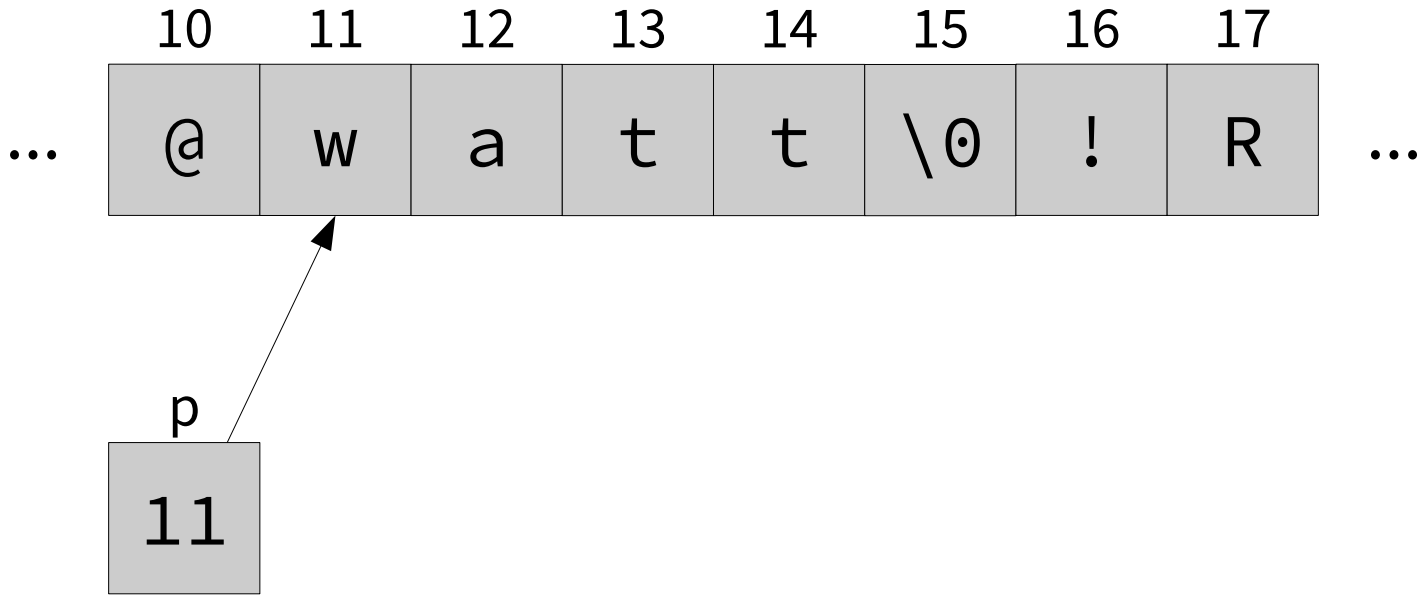
A3




```
printf("%c", *p);
```

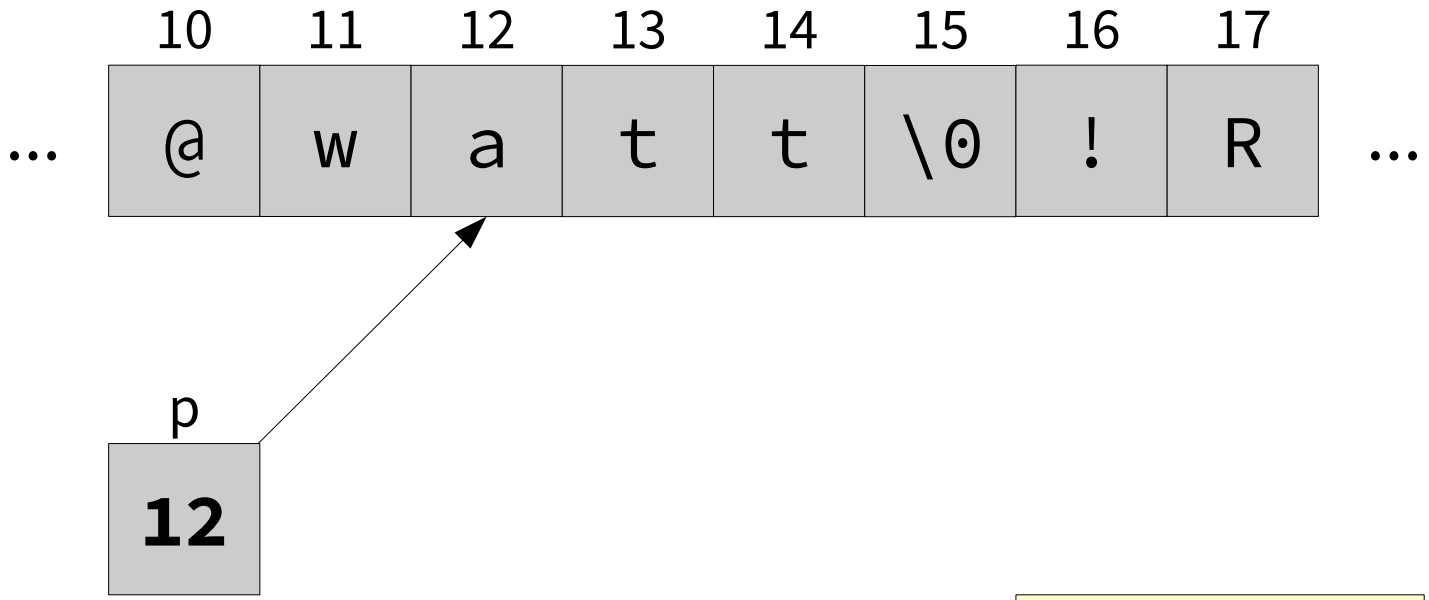
This prints **w**

Q4 - before




```
p++;
printf("%c", *p);
```

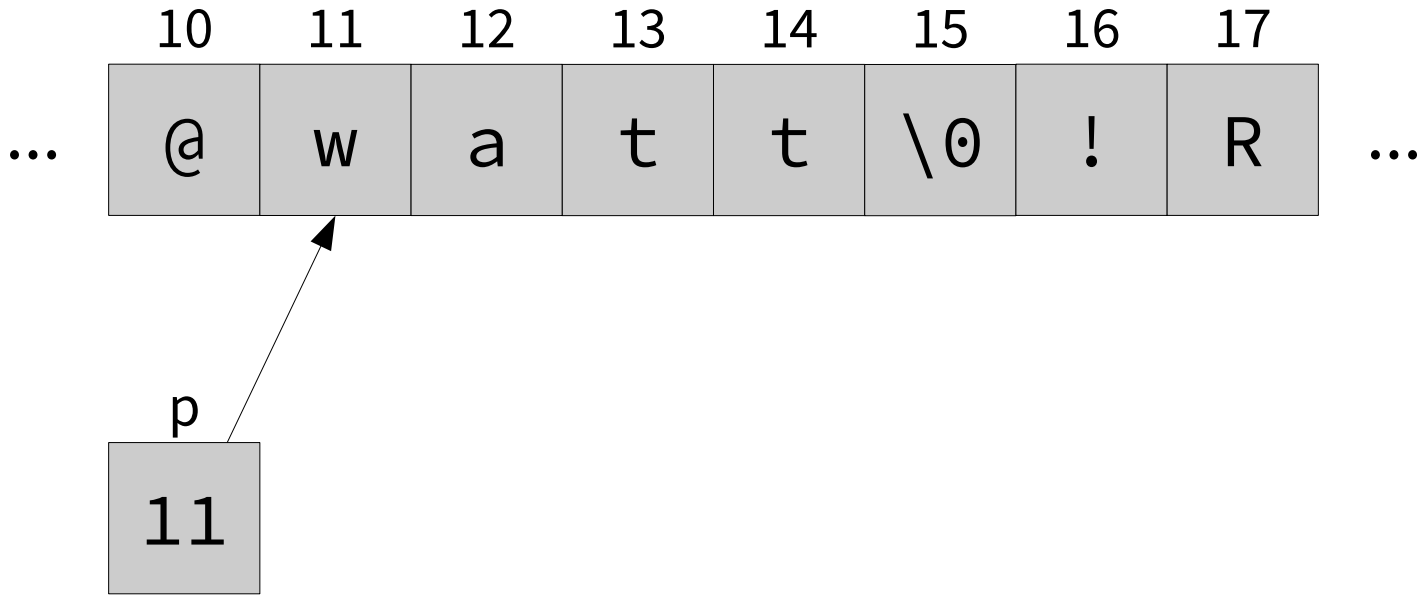
A4 - after




```
p++;
printf("%c", *p);
```

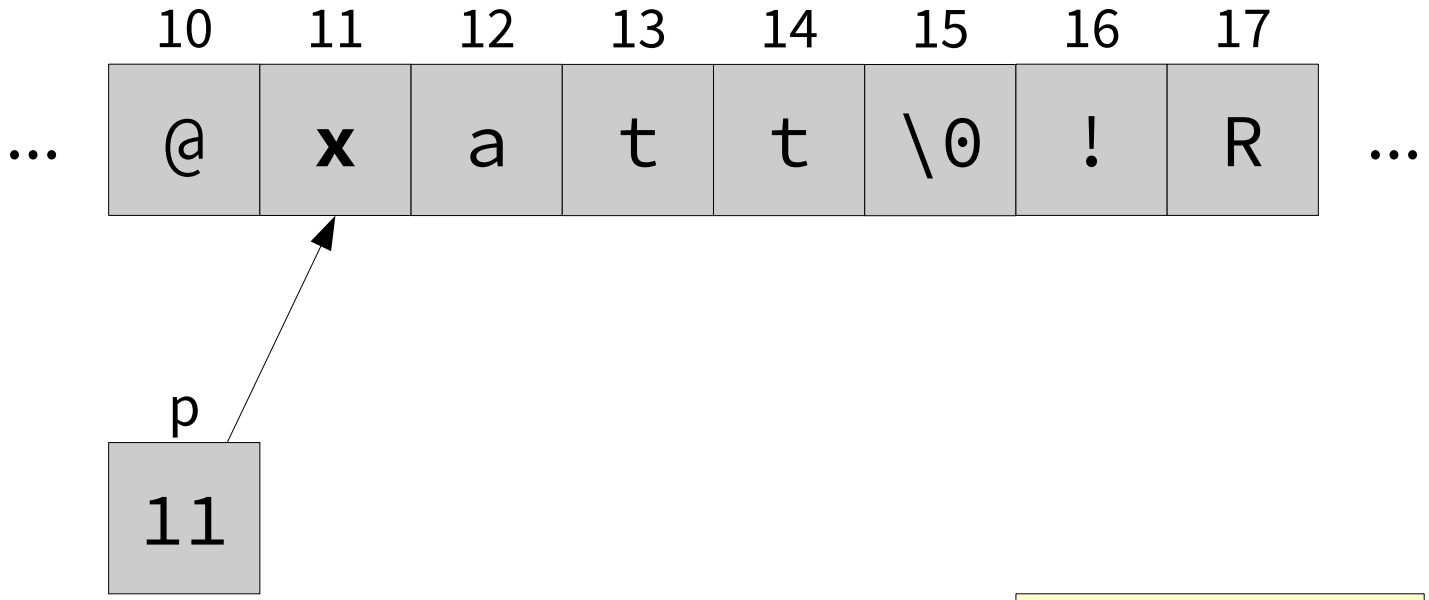
This prints **a**

Q5 - before




```
(*p)++;
printf("%c", *p);
```

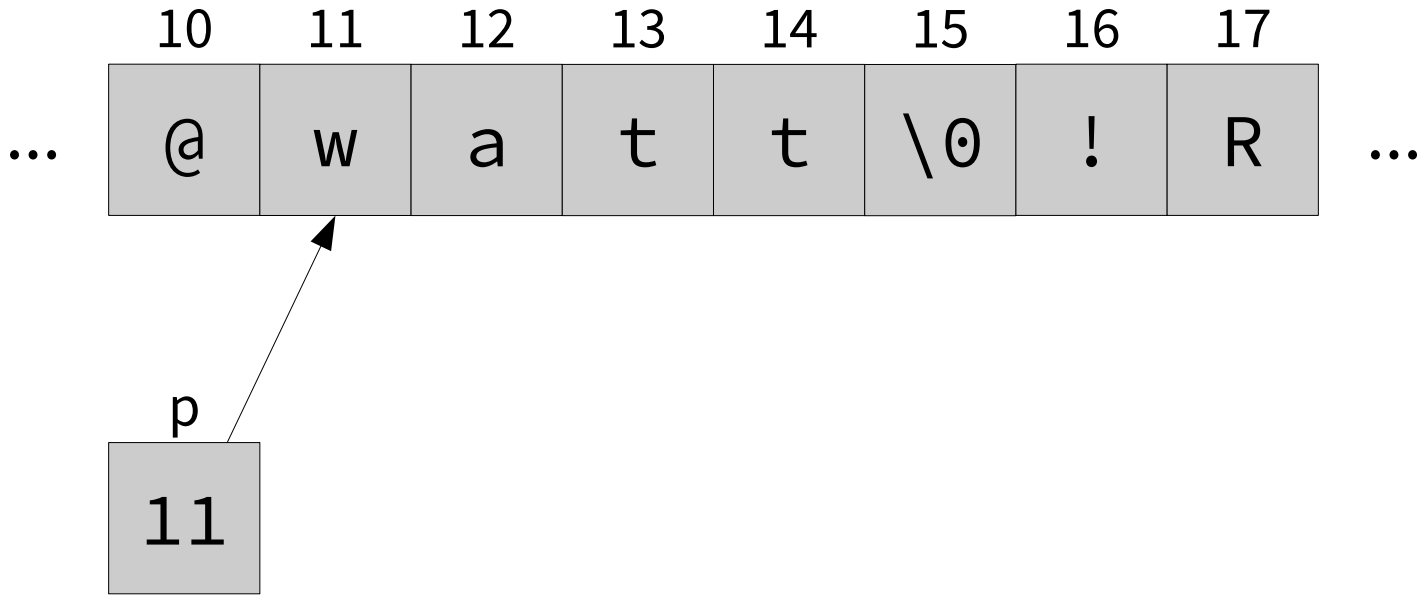
A5 - after




```
(*p)++;
printf("%c", *p);
```

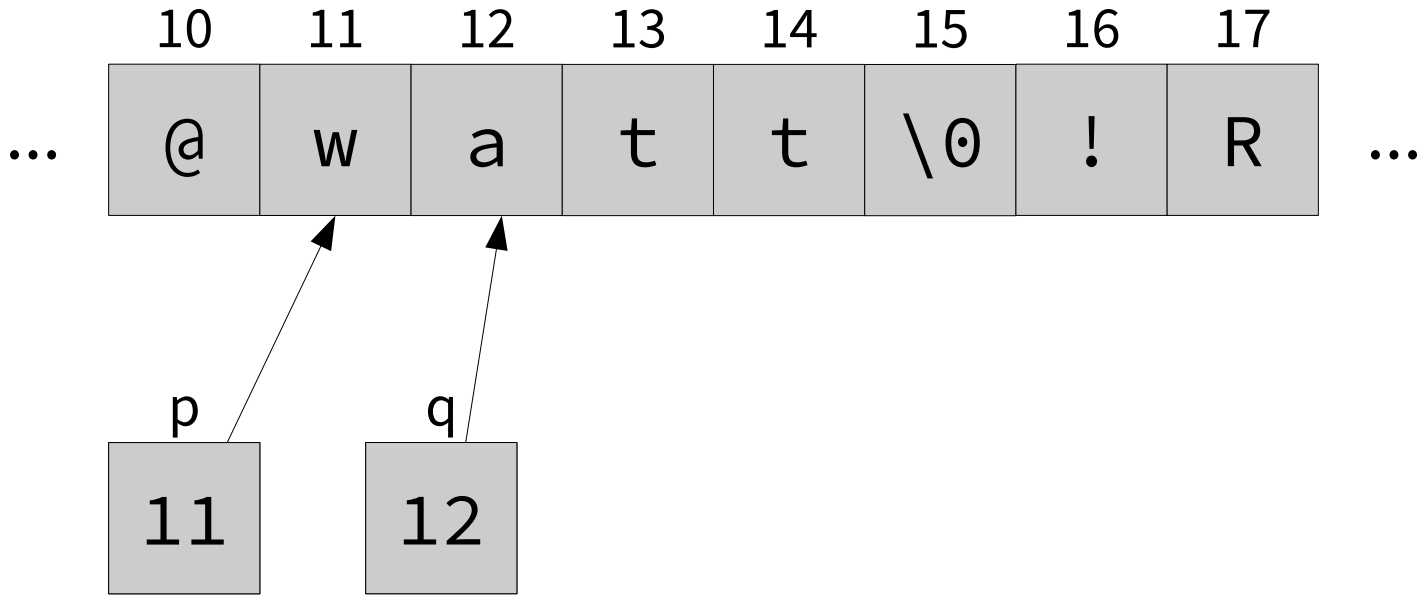
This prints **x**

Q6 - before




```
char *q = p + 1;
printf("%c", *q);
```

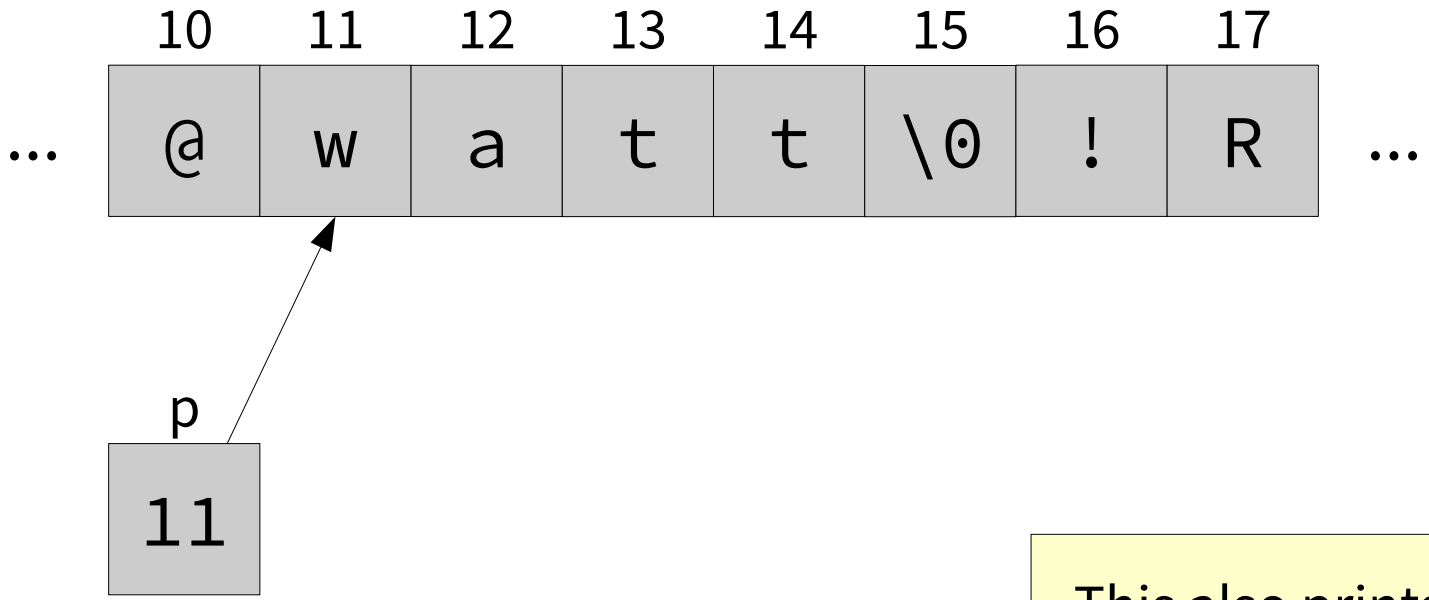
A6 - after




```
char *q = p + 1;
printf("%c", *q);
```

This prints **a**

A6 - simpler

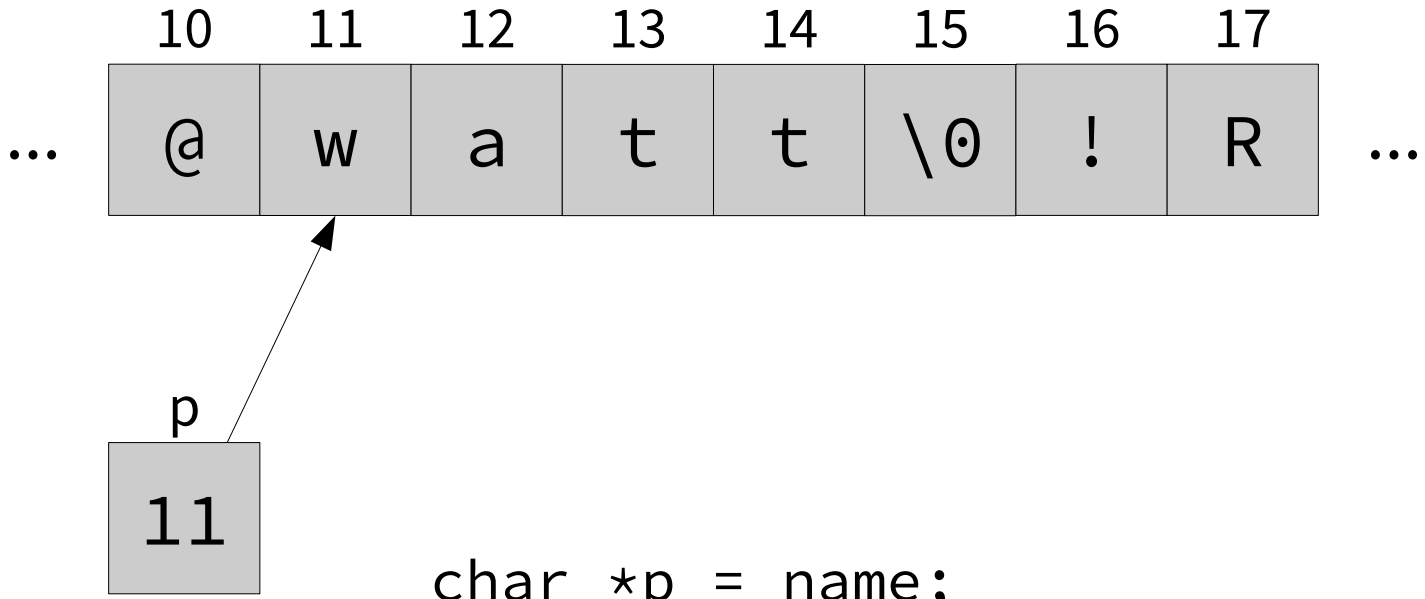



```
printf("%c", *(p + 1));
```

This also prints **a**

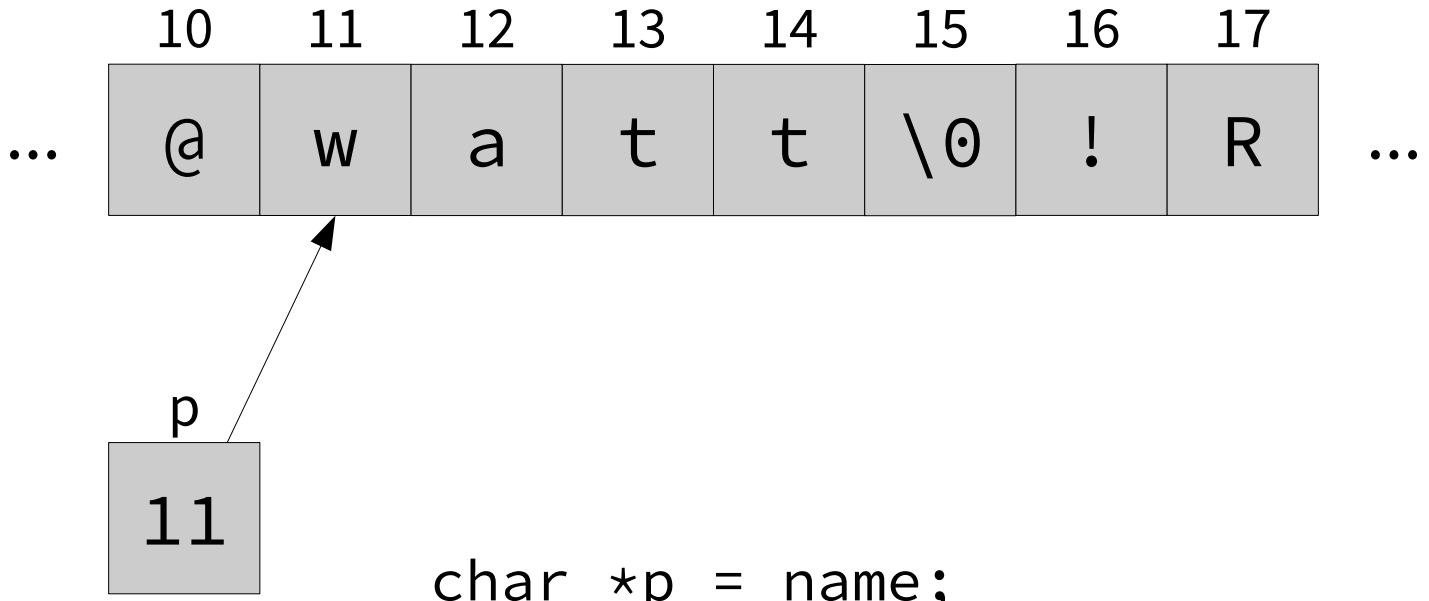
(p + 1 == 12)

Q7 - before




```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

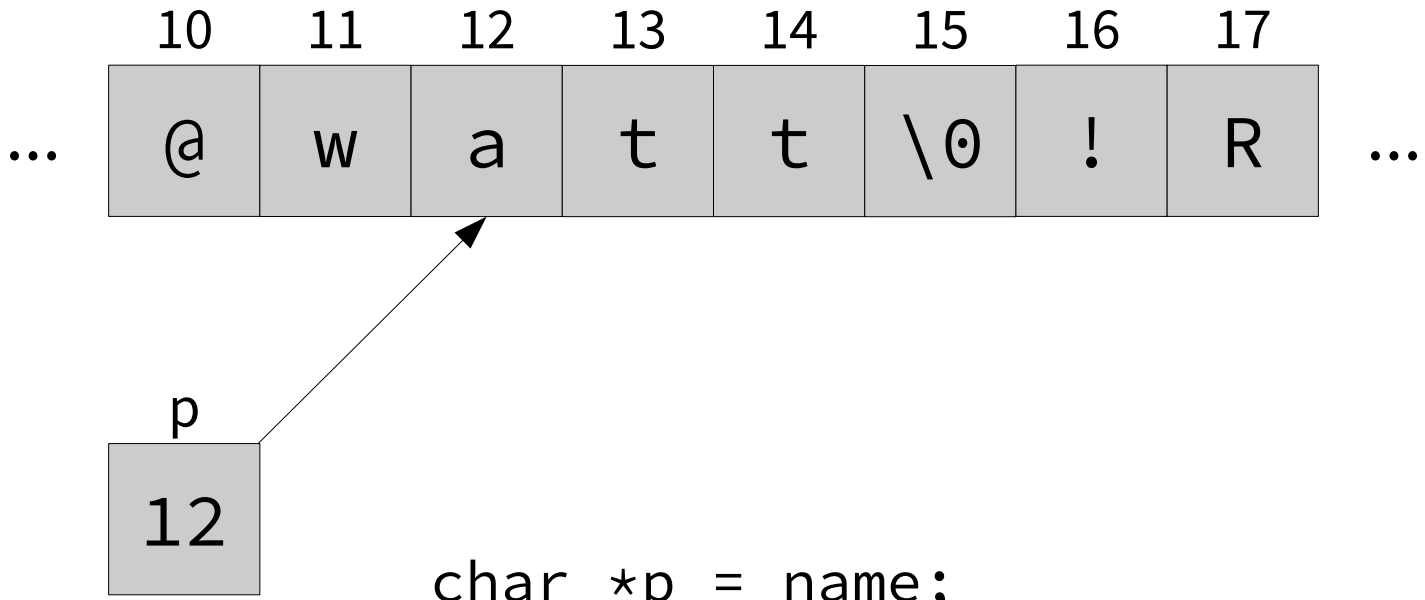
A7 – printing a string




```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

Prints:
w

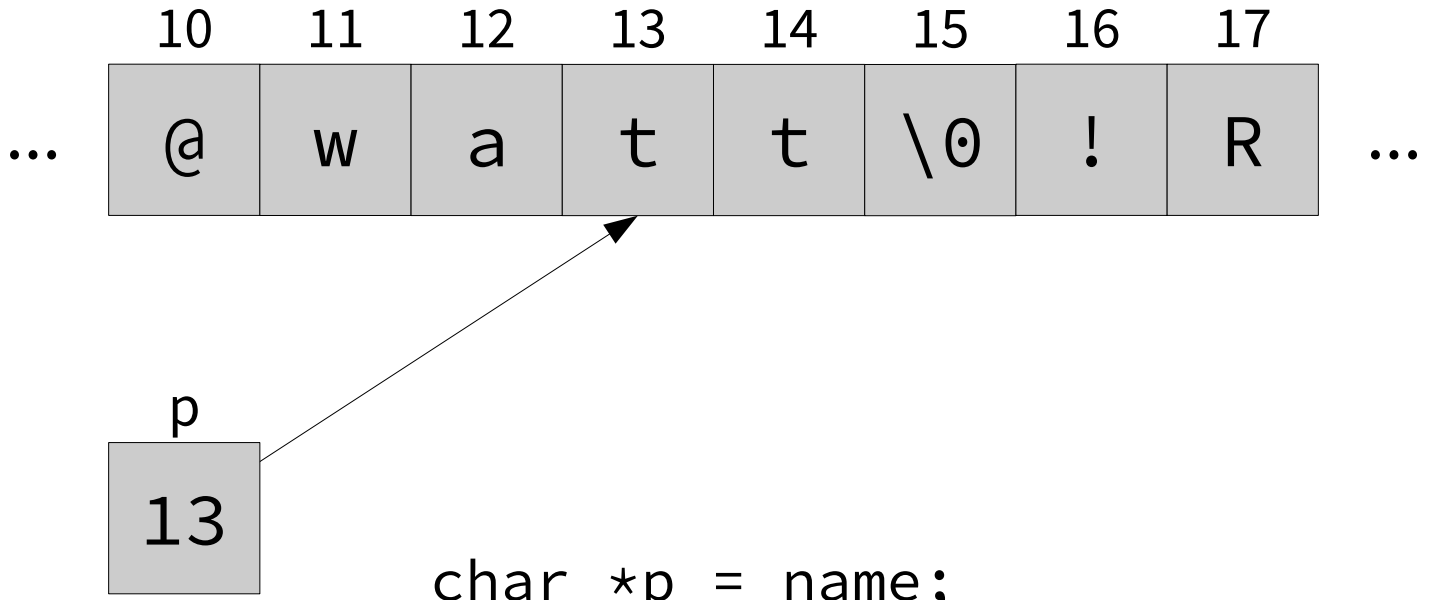
A7 – printing a string




```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

Prints:
wa

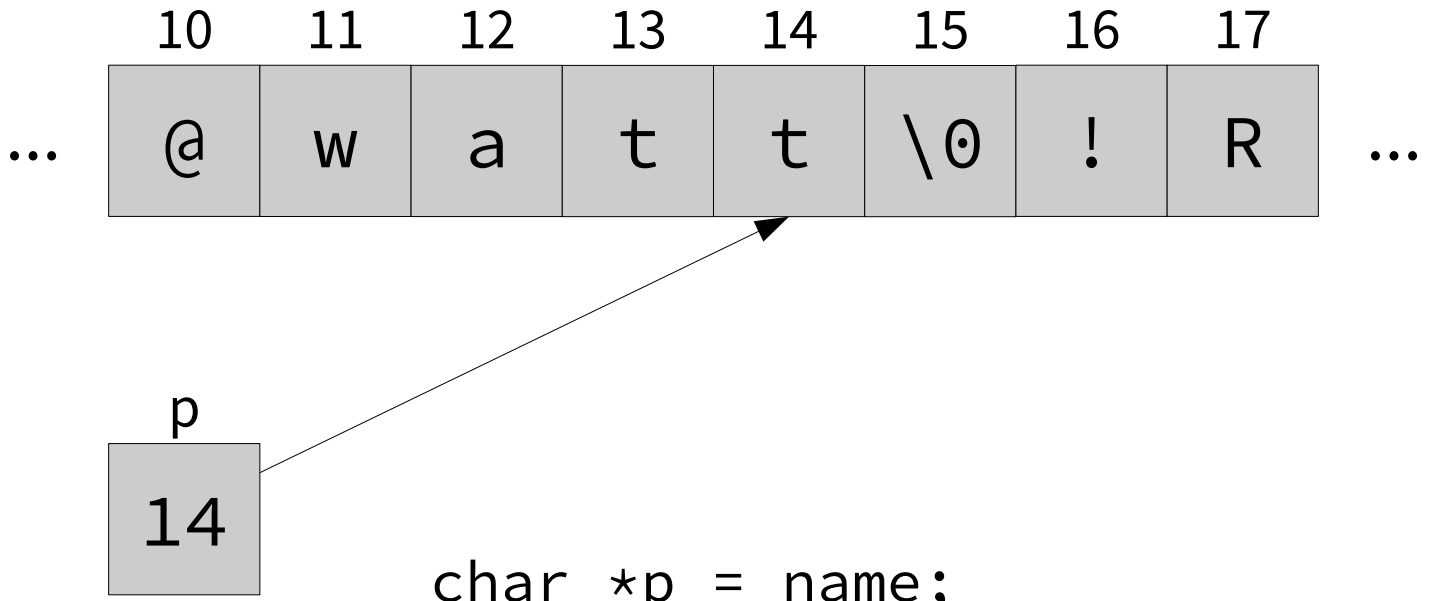
A7 – printing a string




```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

Prints:
wat

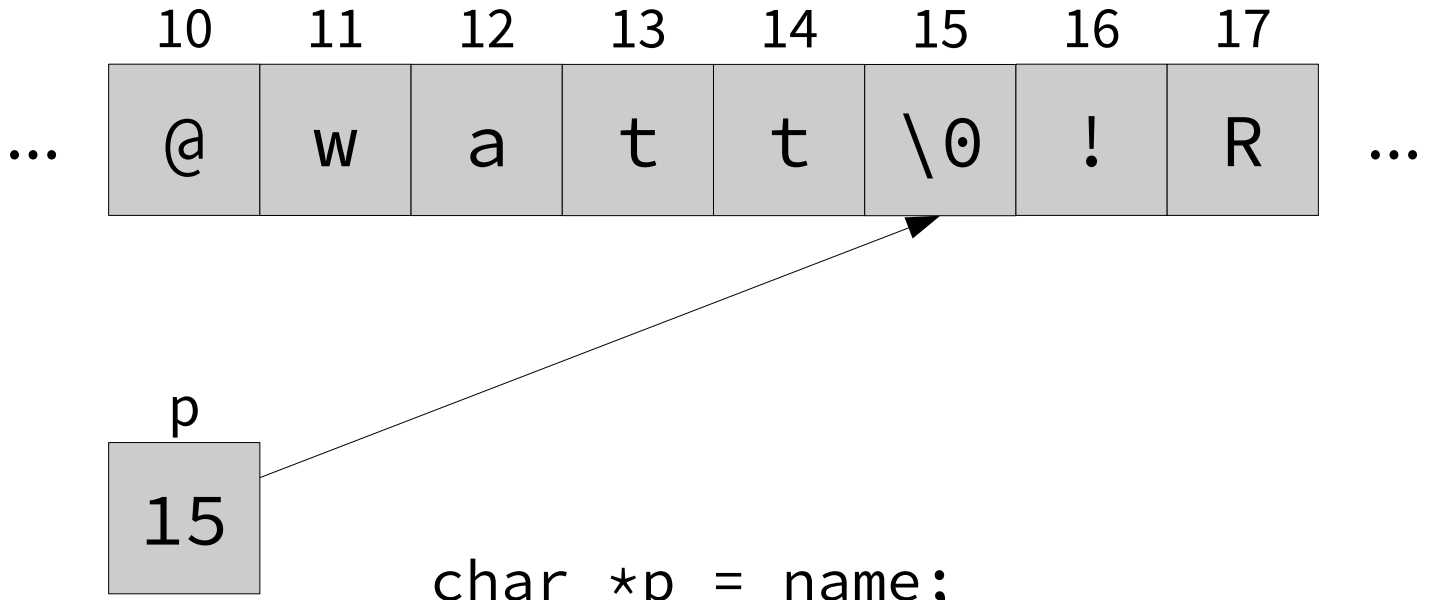
A7 – printing a string




```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

Prints:
watt

A7 – printing a string




```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

Prints:
watt

A7 – shorter

	10	11	12	13	14	15	16	17	
…	@	w	a	t	t	\0	!	R	…

```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

```
char *p = name;
while (*p != 0) {
    printf("%c", *p);
    p++;
}
```

A7 – shorter

	10	11	12	13	14	15	16	17	
…	@	w	a	t	t	\0	!	R	…

```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

```
char *p = name;
while (*p) {
    printf("%c", *p);
    p++;
}
```

A7 – shorter

	10	11	12	13	14	15	16	17	
…	@	w	a	t	t	\0	!	R	…

```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

```
char *p = name;
while (*p)
    printf("%c", *p++);
```

A7 – shorter but wrong

	10	11	12	13	14	15	16	17	
…	@	w	a	t	t	\0	!	R	…

```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

```
char *p = name;
while (*p++)
    printf("%c", *p);
```

What would this do?

A7 – shorter

	10	11	12	13	14	15	16	17	
…	@	w	a	t	t	\0	!	R	…

```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

```
for (char *p = name; *p; p++)
    printf("%c", *p);
```

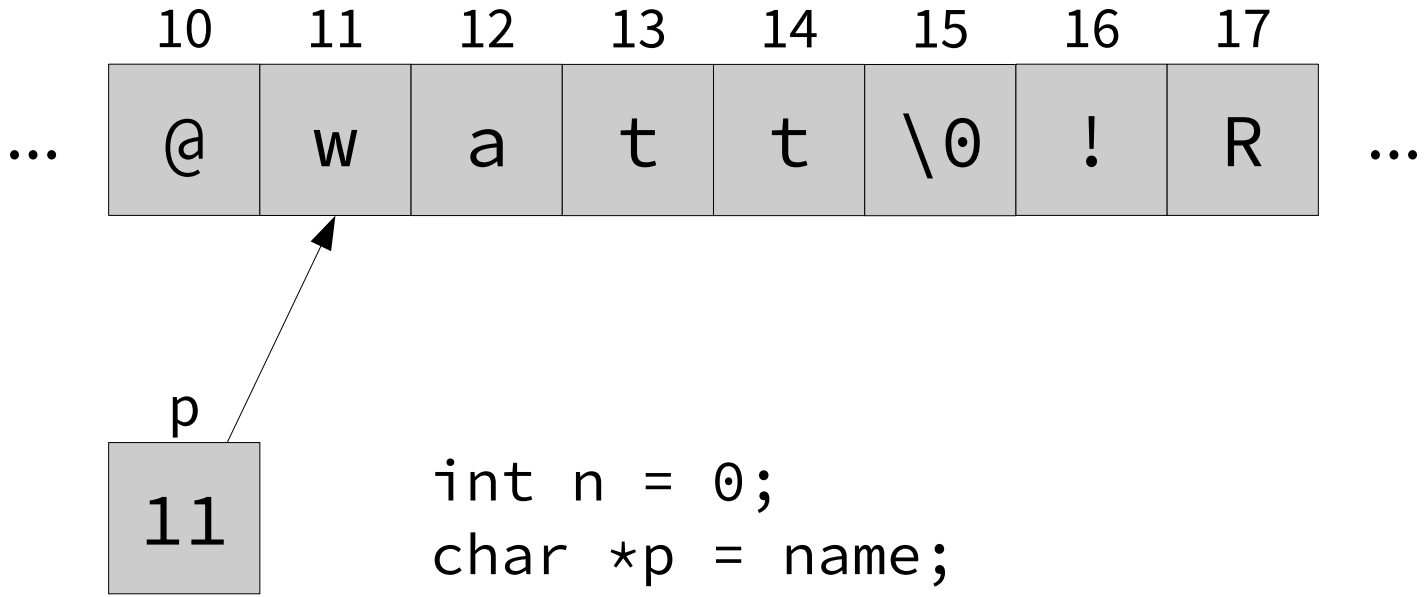
A7 – library functions

	10	11	12	13	14	15	16	17	
…	@	w	a	t	t	\0	!	R	…

```
char *p = name;
while (*p != '\0') {
    printf("%c", *p);
    p++;
}
```

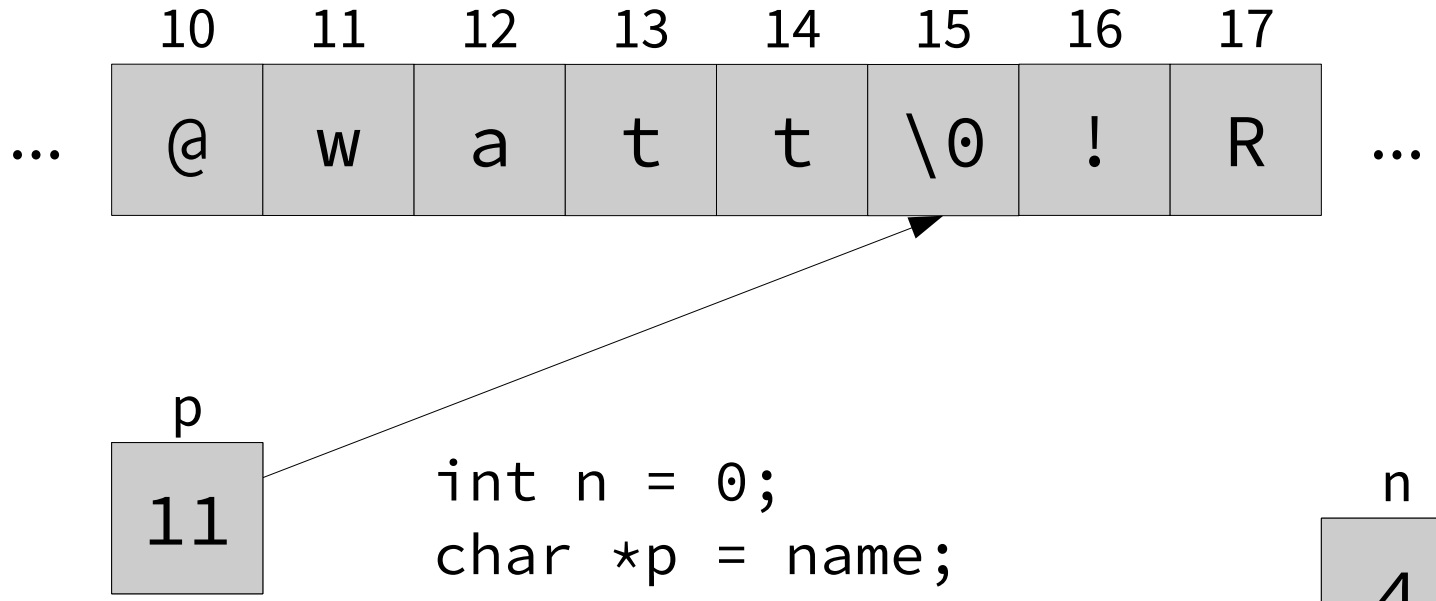
`printf("%s", name);`

Q8 - before

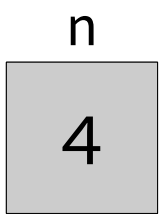



```
int n = 0;
char *p = name;
while (*p++) {
   n++;
}
```

A8 - after




```
int n = 0;
char *p = name;
while (*p++) {
   n++;
}
```



This is **strlen**

Function definitions

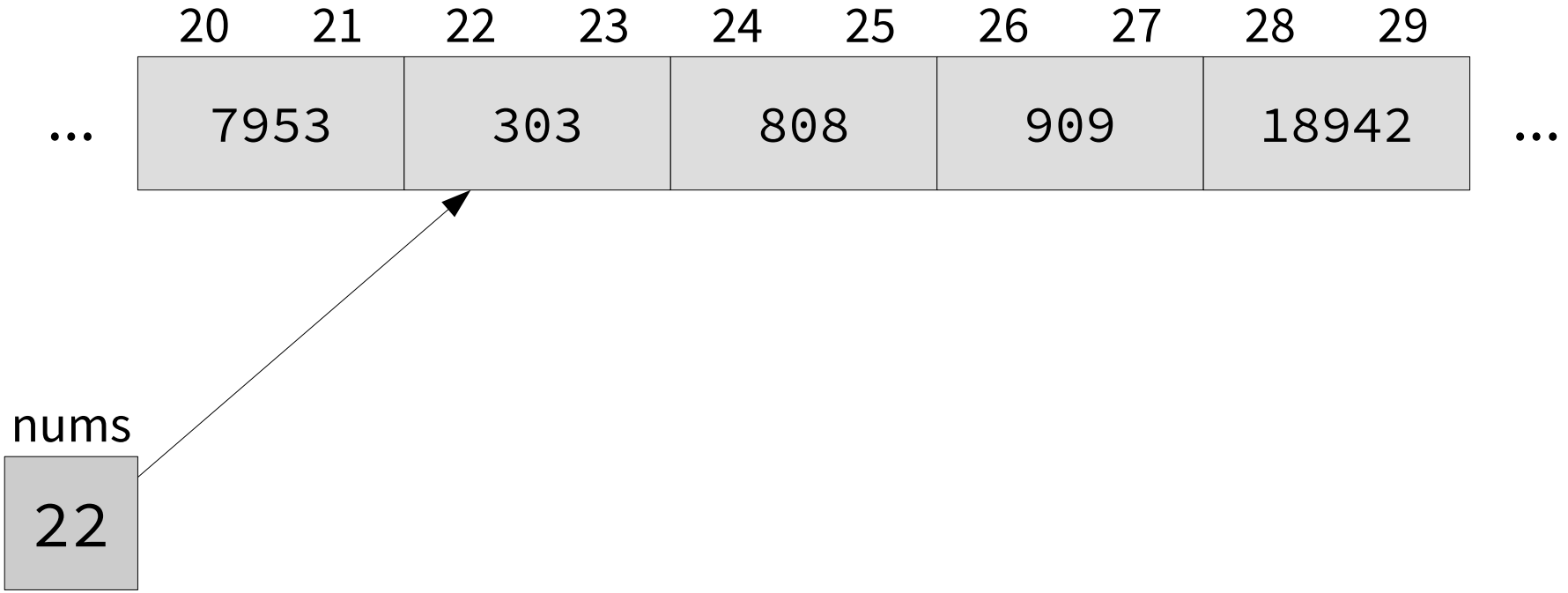
```
void puts(const char *s) {
    while (*s)
        putchar(*s++);
}

size_t strlen(const char *s) {
    size_t n = 0;
    while (*s++)
        n++;
    return n;
}
```

const char *
Read-only pointer to char

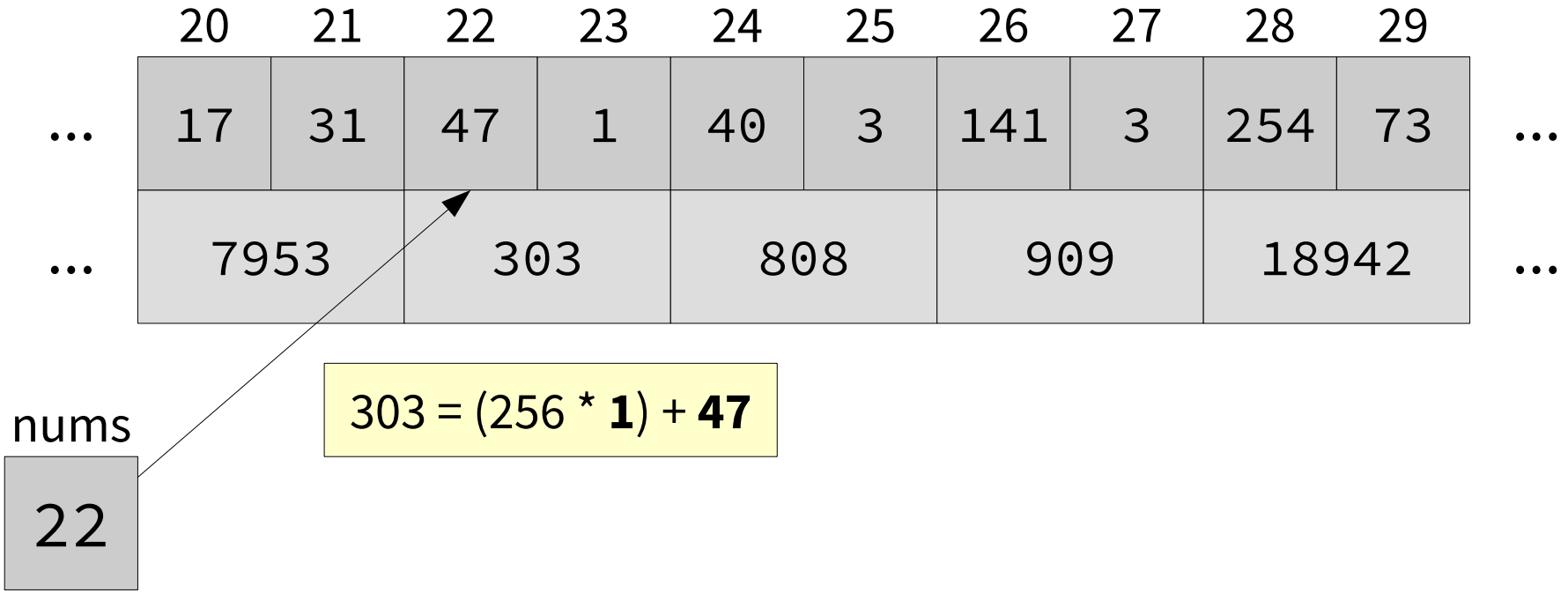
size_t
Unsigned integer type, used to represent sizes of objects in memory (sizeof returns size_t)

Array of 16-bit numbers



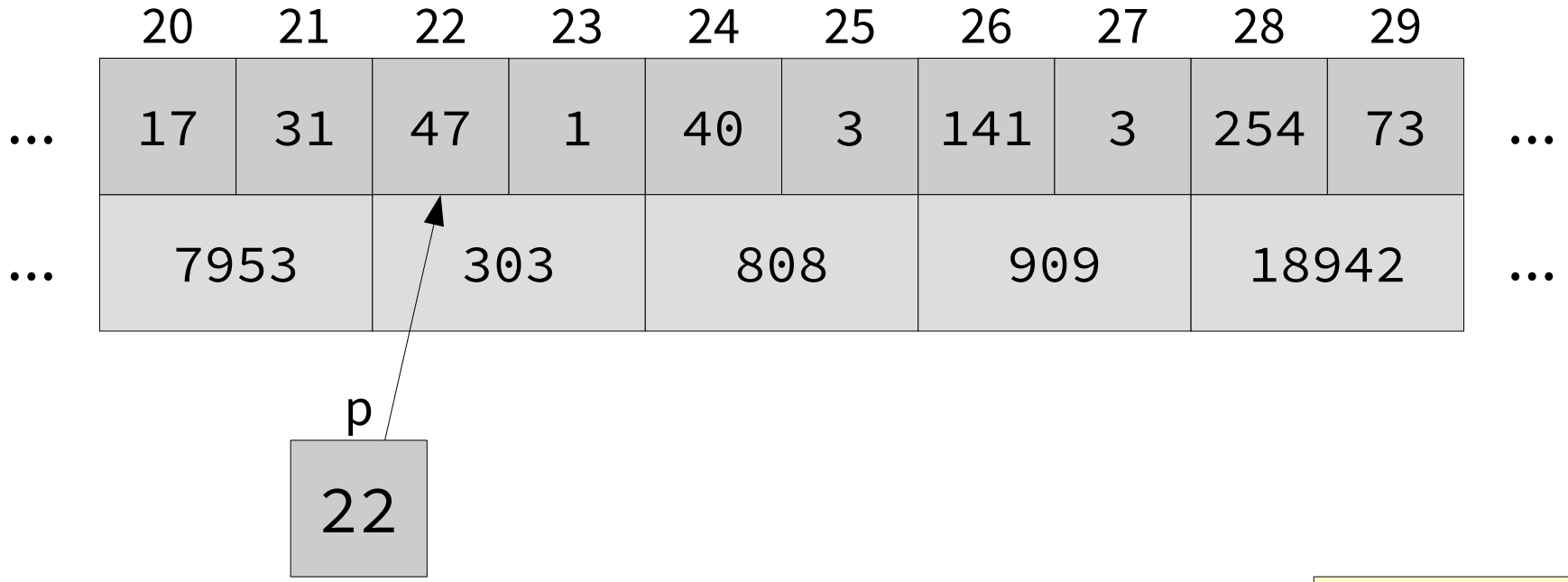

```
int16_t nums = {303, 808, 909};
```

Array of 16-bit numbers




```
int16_t nums = {303, 808, 909};
```

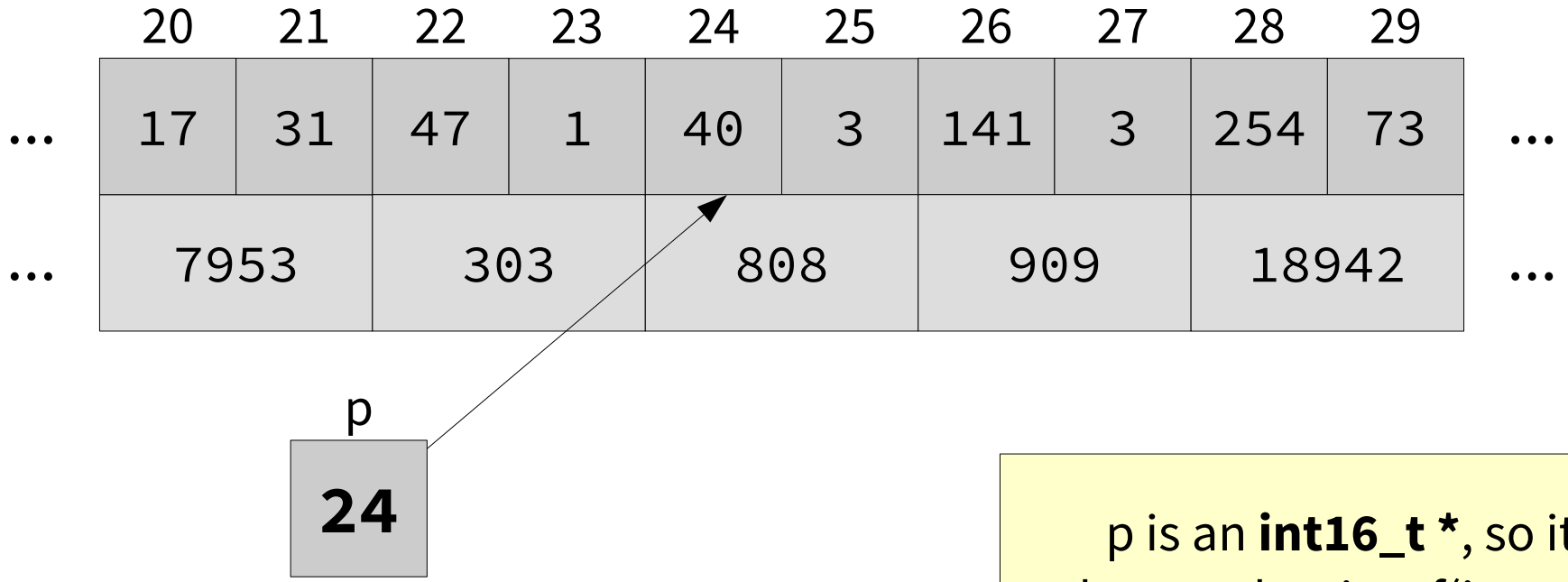
Q9 – before




```
int16_t *p = nums;
p++;
```

What would this do to p?

A9 – after

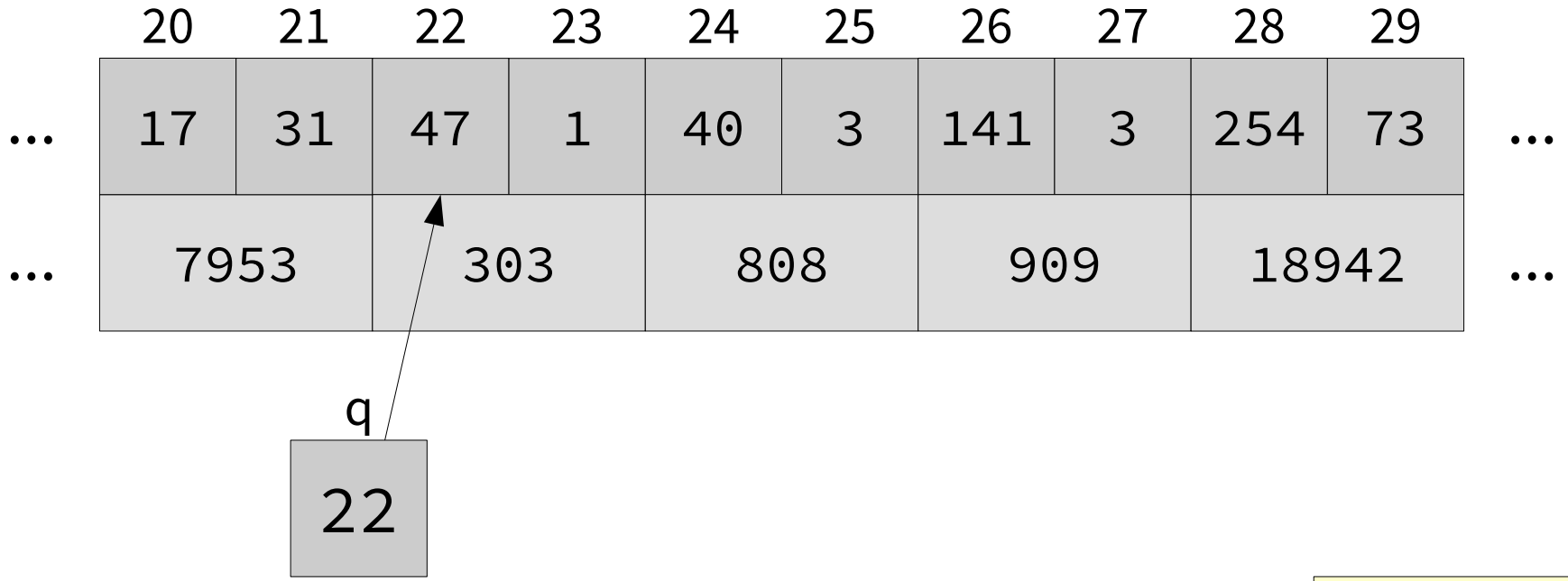



```
int16_t *p = nums;
p++;
```

p is an **int16_t ***, so it advances by sizeof(int16_t).

sizeof(int16_t) == 2

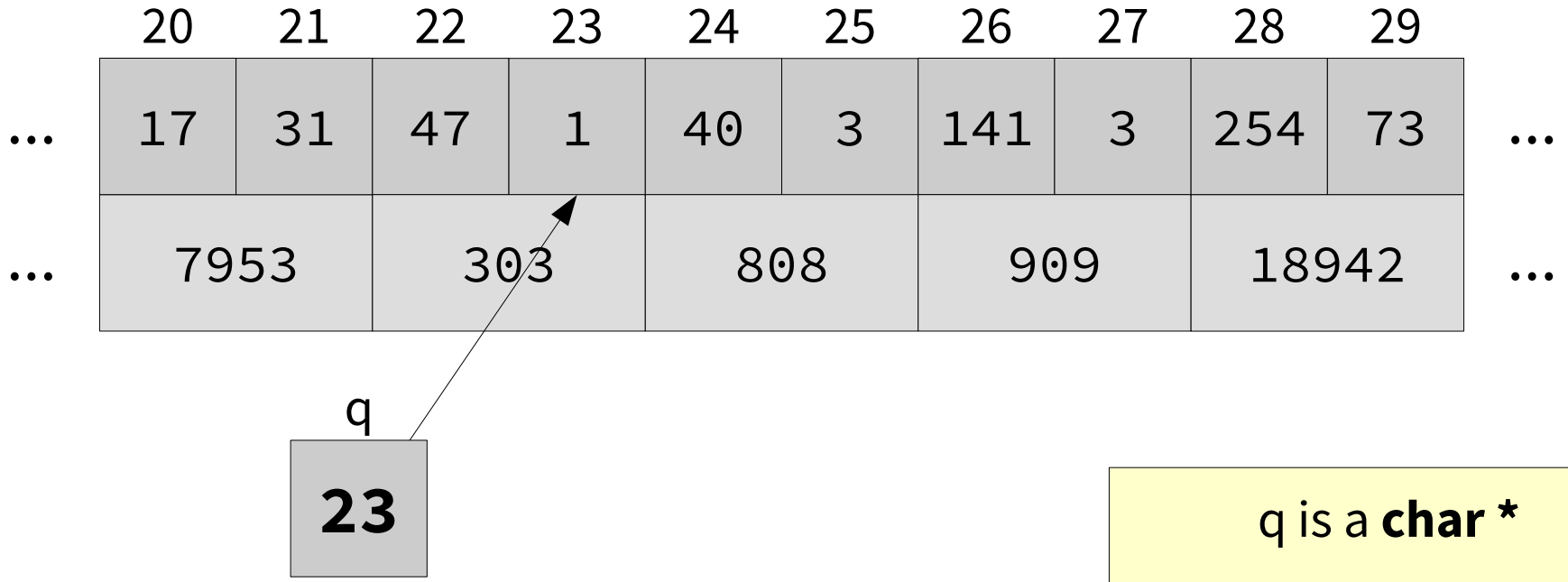
Q10 – before




```
char *q = (char *) nums;
q++;
```

What would this do to q?

A10 – after

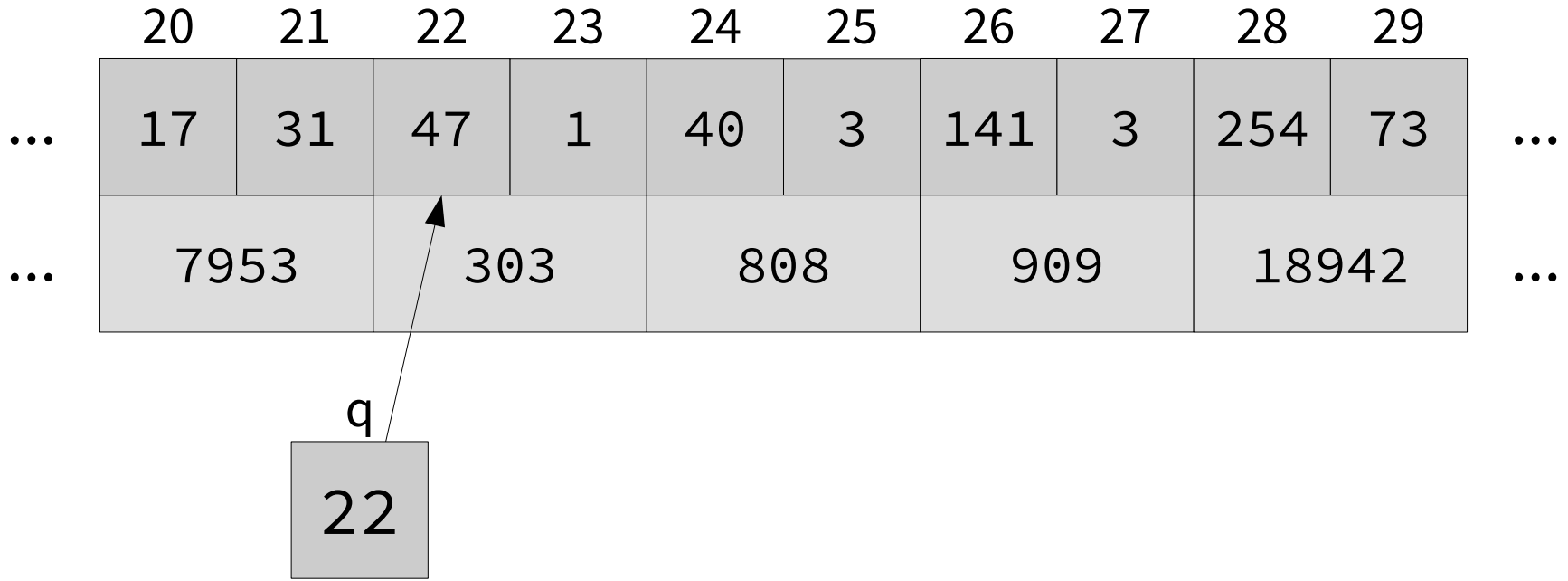



```
char *q = (char *) nums;
q++;
```

q is a **char ***

So it advances by sizeof(char), which is 1

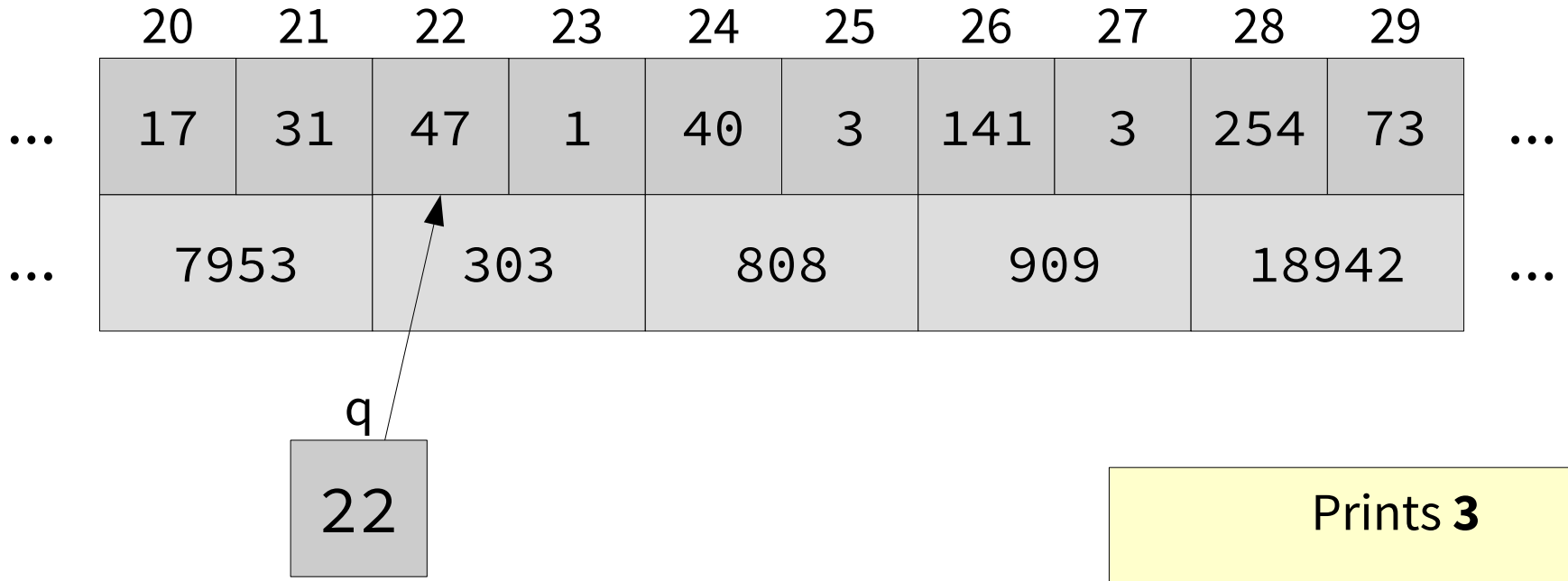
Q11




```
char *q = (char *) nums;
printf("%d", *(q + 3));
```

What would this print?

A11

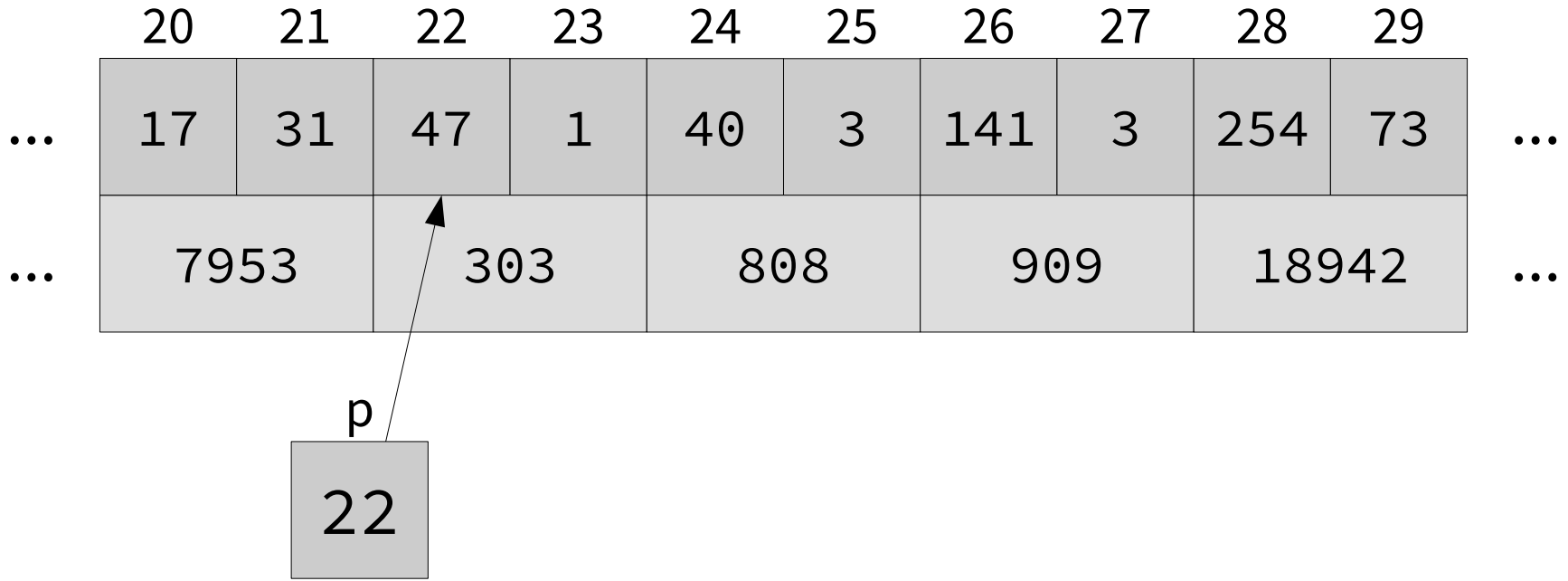



```
char *q = (char *) nums;
printf("%d", *(q + 3));
```

Prints **3**

q is a **char ***, so:
22 + (3 * 1) == 25

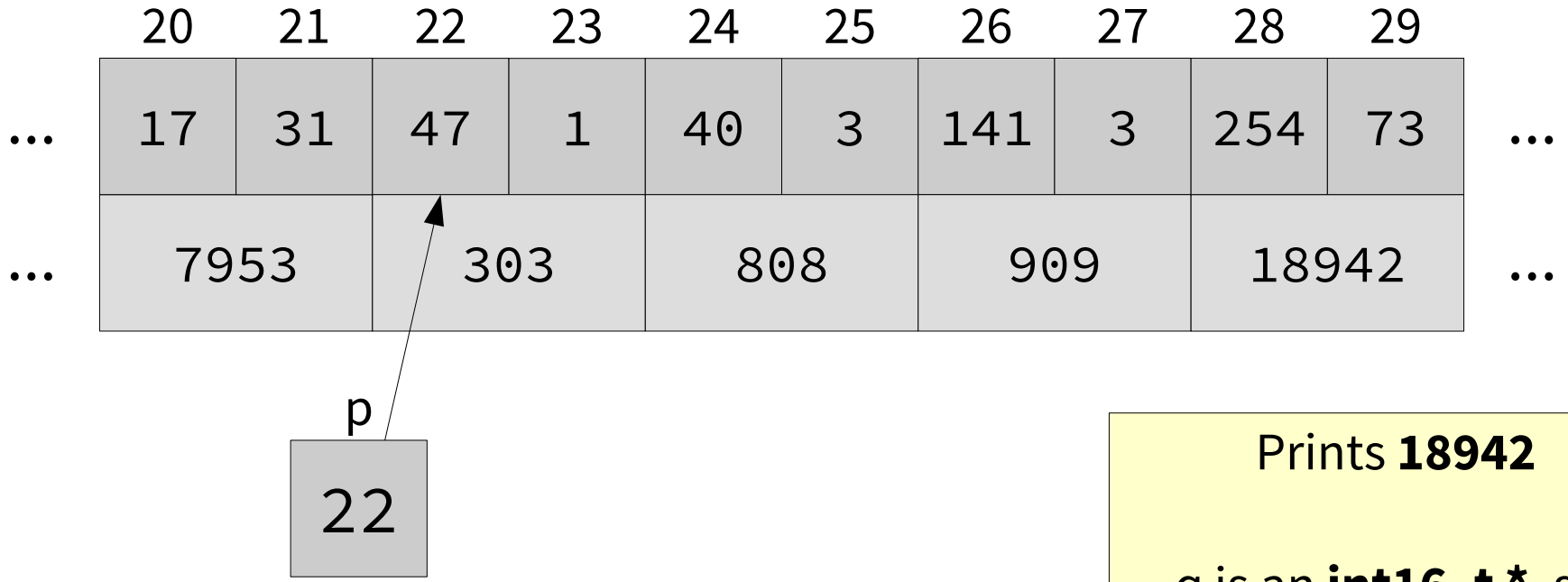
Q12




```
int16_t *p = nums;
printf("%d", *(p + 3));
```

What would this print?

A12




```
int16_t *p = nums;
printf("%d", *(p + 3));
```

Prints **18942**

q is an **int16_t ***, so:
22 + (3 * **2**) == 28
(outside the array!)